



Documentation du Module de Notifications

Projet	Portail PKI - Identité Numérique
Module	Module de Notification
Version	1.0
Date	Avril 2026
Statut	Validé



Table des matières

Table des matières	2
1. Contexte & Architecture	3
2. Prérequis & Configuration	3
2.1 Quelques exemple de client Kafka	3
2.1.1 Java/Springboot	3
2.1.2 JavaScript/NodeJs	4
2.1.3 Php	4
2.2 Configuration pour les Producers	4
2.2.1 Java/Springboot	4
2.2.2 JavaScript/NodeJs	5
2.2.3 Php	5
2.3 Configuration du Consommateur (Interne au Module de Notification)	5
2.4 Les Topics disponibles	6
2.4.1 Note sur la sécurité en Production (SSL/TLS)	7
3. Le modèle DTO	10
3.1 Exemple de structure du JSON attendu	11
4. Mise en œuvre	12
4.1 Java/Springboot	12
4.2 Javascript/Nodejs	12
4.3 Php	13
5. Utilisation des Templates et Variables (HTML)	13
6. Glossaire	14



1. Contexte & Architecture

Ce module a pour objectif de centraliser l'envoi de toutes les notifications (Emails, SMS) générées par les différents microservices de notre stack technologique. Pour garantir de hautes performances et un découplage total entre les services, la communication se fait de manière asynchrone et sécurisée via Apache Kafka.



Figure 1 : Schéma de fonctionnement

NB : Le module de notification n'a pas besoin de savoir quel microservice a envoyé le message, tant que le schéma JSON est respecté.

L'architecture repose sur un découplage strict : les microservices métiers agissent uniquement comme Producteurs (Producers), tandis que le module de notification agit comme unique Consommateur (Consumers).

2. Prérequis & Configuration

Pour qu'un microservice puisse envoyer une notification, il doit configurer un **Producer Kafka**.

2.1 Quelques exemples de client Kafka

Vous devez utiliser une librairie Kafka compatible avec votre langage.

2.1.1 Java/Springboot

Assurez-vous d'avoir la librairie Kafka dans votre pom.xml :

```
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>
```

XML

ou dans votre **build.gradle**



```
implementation 'org.springframework.kafka:spring-kafka'
```

Groovy

2.1.2 JavaScript/NodeJs

Pour javascript, vous pouvez utiliser la librairie **@confluentinc/kafka-javascript** maintenue par Confluence.

2.1.3 Php

En php, vous pouvez vous tourner vers la librairie **php-rdkafka**

2.2 Configuration pour les Producers

Si vous développez un microservice qui doit envoyer des notifications, voici la configuration à appliquer selon votre langage. L'objectif est simple : se connecter au cluster et envoyer un message au format texte (JSON).

2.2.1 Java/Springboot

Dans votre fichier application.yml, vous devez déclarer les sérialiseurs.

```
spring:
  kafka:
    # Les 'bootstrap servers' sont les points d'entrée vers le cluster. Il s'agit
    # des adresses ip de vos brokers.
    bootstrap-servers: localhost:9092,localhost:9094,localhost:9096
    client-id: ${spring.application.name}
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      # On force la conversion de l'objet Java en JSON
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
    properties:
      #Empêche Spring d'envoyer le nom de votre classe Java locale dans Kafka
      spring.json.add.type.headers: false
```

YAML

2.2.2 JavaScript/NodeJs

La configuration est native. Il suffit de déclarer les serveurs (Brokers) lors de l'instanciation de votre client. La sérialisation JSON se fera manuellement dans le code avant l'envoi.



javascript

```
const { Producer } = require('@confluentinc/kafka-javascript');

const producer = new Producer({
  'bootstrap.servers': 'localhost:9092,localhost:9094,localhost:9096',
  'client.id': 'mon-microservice-producer'
});
```

2.2.3 Php

php

```
<?php
```

```
$conf = new \RdKafka\Conf();

// Configuration des bootstrap servers (une seule chaîne séparée par des virgule:
$conf->set('metadata.broker.list', 'localhost:9092,localhost:9094,localhost:9096

// Identification du client
$conf->set('client.id', 'mon-microservice-producer');

// Instanciation du Producer
$producer = new \RdKafka\Producer($conf);
```

2.3 Configuration du Consommateur (Interne au Module de Notification)

Cette section est documentée à titre informatif pour comprendre comment le module de notification traite vos messages.

Le module de notification utilise une configuration Spring Kafka avancée pour se protéger des erreurs de formatage et mapper dynamiquement le JSON entrant vers son propre modèle de données.



YAML

```
spring:
  kafka:
    bootstrap-servers: ${IP_ADRESS}:${PORT}
    consumer:
      group-id: notification-service-group
      # S'il n'y a pas d'historique de lecture, on commence par le plus ancien
      # message non lu
      auto-offset-reset: earliest

      # Protection contre les "Poison Pills" (Messages corrompus)
      # Au lieu de crasher en boucle infinie sur un mauvais JSON, cette classe
      # intercepte l'erreur
      key-deserializer:
        org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
      value-deserializer:
        org.springframework.kafka.support.serializer.ErrorHandlingDeserializer

      properties:
        # Sérialiseurs réels appelés si le message est valide
        spring.deserializer.key.delegate.class:
          org.apache.kafka.common.serialization.StringDeserializer
        spring.deserializer.value.delegate.class:
          org.springframework.kafka.support.serializer.JsonDeserializer

        # Mapping dynamique du JSON vers le DTO interne
        spring.json.trusted.packages: "path_to_dto_folder"
        spring.json.use.type.headers: false # On ignore la classe d'origine du
        # Producteur
        spring.json.value.default.type: "path_to_dto" # On force la conversion
        # vers cette classe exacte
```

2.4 Les Topics disponibles

Le module de notification écoute actuellement sur les topics suivants

- notify.sms
- notify.email
- notify.websocket

La convention retenue pour la nomenclature exige que le topic débute par "notify." Les topics sont déclarés dans le consumer du module de notification.



2.5 Note sur la sécurité en Production (SSL/TLS)

Que vous soyez Producteur ou Consommateur, les environnements de production nécessitent une connexion sécurisée au cluster Kafka.

Java/Springboot

Les propriétés suivantes devront être activées via vos variables d'environnement (application.yml).

YAML

```
spring:
  kafka:
    properties:
      security.protocol: SSL
    ssl:
      key-store-location: classpath:path_to_keystore.jks
      key-store-password: ${KEYSTORE_PASSWORD}
      key-password: ${KEY_PASSWORD}
      key-store-type: PKCS12
      trust-store-location: classpath:path_to_truststore.jks
      trust-store-password: ${TRUSTSTORE_PASSWORD}
      trust-store-type: JKS
```




JavaScript/NodeJs

Contrairement à Java qui utilise des chemins vers des fichiers "stores", **@confluentinc/kafka-javascript** attend le contenu des fichiers sous forme de texte ou de Buffer.

javascript

```
const { Producer } = require('@confluentinc/kafka-javascript');

const producer = new Producer({
  // 1. Connexion de base
  'bootstrap.servers': 'kafka-1:9093,kafka-2:9093',
  'client.id': 'mon-service-enrolement',

  // 2. Activation du SSL
  'security.protocol': 'ssl',

  // 3. Chemins vers les certificats (Fichiers sur le disque)
  'ssl.ca.location': './certs/ca.pem',
  'ssl.certificate.location': './certs/client-cert.pem',
  'ssl.key.location': './certs/client-key.pem',

  // 4. Mot de passe de la clé (si nécessaire)
  'ssl.key.password': 'votre_mot_de_passe_cle',

  // Optionnel : vérifier que le nom du serveur correspond au certificat
  'enable.ssl.certificate.validation': 'true'
});

// Connexion explicite requise en mode natif
producer.connect();
```




Php

La librairie PHP rdkafka utilise des chemins absolus vers les fichiers sur le disque.

php

```
$conf = new \RdKafka\Conf();
$conf->set('metadata.broker.list', 'kafka-1:9093,kafka-2:9093');
$conf->set('security.protocol', 'ssl');

// Chemins vers les fichiers sur le disque
$conf->set('ssl.ca.location', './certs/ca.pem');
$conf->set('ssl.certificate.location', './certs/client-cert.pem');
$conf->set('ssl.key.location', './certs/client-key.pem');
$conf->set('ssl.key.password', 'votre_mot_de_passe_cle');

$producer = new \RdKafka\Producer($conf);
```



3. Le modèle DTO

Les microservices ne partagent pas de librairie Java commune. La communication est **agnostique**. Votre microservice doit simplement envoyer un objet qui se transforme en ce JSON exact.

Champ	Type	Requis	Description
subject	String	☒	L'objet de l'email ou l'en-tête du SMS.
platform	Enum	☒	La plateforme d'origine (ex: PORTAL, SANDBOX).
type	String	☒	Catégorisation interne de l'événement (ex: API_RESPONSE).
primaryChannel	Enum	☒	Le canal d'envoi principal. Valeurs supportées : EMAIL, SMS, WEBSOCKET.
secondaryChannel	Enum	☒	Le canal de secours si le principal échoue.
fallbackDelay	Integer	☒	Délai en secondes avant de déclencher le secondaryChannel.
visibleInInterface	Boolean	☒	Si true, la notification sera visible dans l'historique in-app (Défaut: true).
templateName	String	☒	Le nom exact du fichier HTML à utiliser pour le rendu.
variables	Object	☒	Variables globales (paires clé-valeur) injectées pour tous les destinataires.
recipients	Array	☒	Liste des destinataires. Doit contenir au moins un objet.
recipients[].email	String	*	Obligatoire si le canal (primary ou secondary) est EMAIL.
recipients[].phoneNumber	String	*	Obligatoire si le canal (primary ou secondary) est SMS.
recipients[].personalVariables	Object	☒	Variables spécifiques injectées uniquement pour ce destinataire



3.1 Exemple de structure du JSON attendu

```
JSON
{
  "subject": "Titre de l'email ou du SMS",
  "platform": "PORTAL",
  "type": "API_RESPONSE",
  "primaryChannel": "EMAIL",
  "secondaryChannel": "SMS",
  "fallbackDelay": 300,
  "visibleInInterface": true,
  "templateName": "welcome-package",
  "variables": {
    "globalVar1": "Valeur commune à tous les destinataires"
  },
  "recipients": [
    {
      "email": "jean.dupont@example.com",
      "phoneNumber": "+22990000000",
      "personalVariables": {
        "name": "Jean Dupont",
        "otp": "123456"
      }
    }
  ]
}
```



4. Mise en œuvre

Voici comment publier le JSON vers Kafka selon votre stack technique.

4.1 Java/Springboot

Java

```
// Construction via votre DTO local (qui sera converti en JSON par Spring)
NotificationEvent event = NotificationEvent.builder()
    .subject("Alerte Sécurité")
    .platform("PORTAL")
    .primaryChannel("EMAIL")
    .templateName("alert.html")

    .recipients(List.of(NotificationRecipientDto.builder().email("admin@test.com").build()))
    .build();

kafkaTemplate.send("notify.security.events", event);
```

4.2 Javascript/Nodejs

JavaScript

```
const payload = {
  subject: "Alerte Sécurité",
  platform: "PORTAL",
  primaryChannel: "EMAIL",
  templateName: "alert.html",
  recipients: [{ email: "admin@test.com" }]
};

await producer.send({
  topic: 'notify.security.events',
  messages: [
    { value: JSON.stringify(payload) },
  ],
});
```



4.3 Php

```
PHP

$payload = [
    "subject" => "Alerte Sécurité",
    "platform" => "PORTAL",
    "primaryChannel" => "EMAIL",
    "templateName" => "alert.html",
    "recipients" => [["email" => "admin@test.com"]]
];

$topic = $producer->newTopic("notify.security.events");
$topic->produce(RD_KAFKA_PARTITION_UA, 0, json_encode($payload));
$producer->poll(0);
```

5. Utilisation des Templates et Variables (HTML)

Le module de notification utilise un moteur de rendu Thymeleaf pour générer des emails à partir de fichiers HTML contenant des variables dynamiques.

Processus actuel de création

- Création du fichier HTML : Le fichier doit être placé dans le dossier **src/main/resources/templates/** du module de notification.
- Utilisation des variables : Dans le HTML, vous pouvez utiliser la syntaxe du moteur de template (ex: **{{ name }}** ou **th:text="\${name}"**) pour afficher les données provenant des champs variables ou **personalVariables** de votre JSON. Vous n'avez pas besoin d'effectuer une boucle sur ces champs dans votre template. Le module fusionne automatiquement les variables et les personalVariables du destinataire en cours.
- Si vous transmettez une liste d'actions, vous devez utiliser la directive **th:each** de Thymeleaf pour itérer dessus et générer les éléments HTML correspondants.
- Appel : Dans votre code source, renseignez simplement le nom du fichier dans le champ **templateName** (ex: templateName: "GENERIC_ACTION")
- Voici un exemple de template créé pour une action générique :



```
generic-action.html
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns:th="http://www.thymeleaf.org" xmlns="http://www.w3.org/1999/xhtml">
<head>
<title th:text="${subject}">Notification</title>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1.0"/>
<link href="https://fonts.googleapis.com/css2?family=Montserrat:wght@400;600;700&display=swap" rel="stylesheet">
<style>
body { font-family: "Montserrat", sans-serif; font-size: 16px; color: #333; line-height: 1.4; }
.container { max-width: 600px; margin: 0 auto; padding: 24px; }
.content-area { padding: 24px 0; }
.center-item { text-align: center; margin: 32px 0; }
.btn { padding: 12px 24px; border-radius: 6px; font-weight: 600; color: #ffffff !important; text-decoration: none; display: inline-block; }
.fallback-link { font-size: 12px; color: #666; word-break: break-all; margin-top: 16px; }
.divider { border-top: 1px solid #eee; margin: 32px 0; }
</style>
</head>
<body>
<div class="container">
<h3 th:style="color: ' + (${themeColor} ? : '#333') " th:text="${platformName ? : 'PKI ASIN'}">Plateforme</h3>

<div class="content-area">
<p>Bonjour <span th:text="${firstName}"></span></p>

<div th:utext="${content}">
</div>

<div th:if="${actions != null and !actions.isEmpty()}" class="center-item">
<th:block th:each="action : ${actions}">
<a class="btn" th:style="background-color: ' + (${themeColor} ? : '#225887') + ';"
th:href="${action.targetUrl}" th:text="${action.label}">Bouton d'action</a>

<div class="fallback-link">
Ou copiez ce lien : <a th:href="${action.targetUrl}" th:text="${action.targetUrl}"></a>
</div>
</th:block>
</div>

<p>S'il s'agit d'une erreur, vous pouvez ignorer ce message.</p>
</div>

<div class="divider"></div>
<p style="text-align: center;">Cet e-mail vous a été envoyé automatiquement.</p>
</div>
</body>
</html>
```

6. Glossaire

Kafka: Plateforme distribuée de diffusion de données en continu permettant de publier, stocker, traiter et souscrire à des flux d'évènements en temps réel.

Producteurs (Producers): Service qui publie des messages dans Kafka.

Consommateurs (Consumers): Service qui lit et traite les messages publiés.

Topic: Canal thématique dans lequel les messages sont publiés et organisés.